CYBER SECURITY

Practical Web Application Security Assessment

Assignment 1

Course	**Cyber Security**
Program	**BS Computer Science**
Semester	**5th**
Instructor	**Mahek Kausar**
Assignment Title	**Practical Web Application Security Assessment Using Altoro Mutual**
Target Application	**Altoro Mutual (demo banking site) — https://demo.testfire.net/**
Total Marks	**100**
Submission Format	**PDF report with screenshots**
Due Date	**27-10-2026**

Submitted By

Muhammad Imran Butt: L1F24BSCS0546

Abdul Rehman: L1F24BSCS0011

Haseeb Soban: L1F24BSCS0555

Faiz Ahmad Ijaz: L1F24BSCS0775

1. Introduction and Objectives

Web application security is an important aspect of protecting user information, preventing unauthorized access, and ensuring the confidentiality and integrity of data. Vulnerabilities in web applications can allow attackers to manipulate database queries, execute malicious scripts in users' browsers, or gain access to resources without proper authorization.

The purpose of this assessment is to evaluate the security of the **Altoro Mutual demo web application** (https://demo.testfire.net/) by conducting controlled security tests on selected features. The assessment focuses on two common web application vulnerabilities: **SQL Injection (SQLi)** and **Cross-Site Scripting (XSS)**.

The main objectives of this assessment are:

- To explore the website and identify relevant features for security testing.
- To investigate the Sign In functionality for potential SQL injection vulnerabilities.
- To examine the Search functionality for potential cross-site scripting vulnerabilities.
- To document test inputs, observed application responses, and supporting screenshots.
- To assess the potential security impact of any confirmed vulnerabilities.
- To recommend appropriate security measures to prevent or reduce these vulnerabilities.

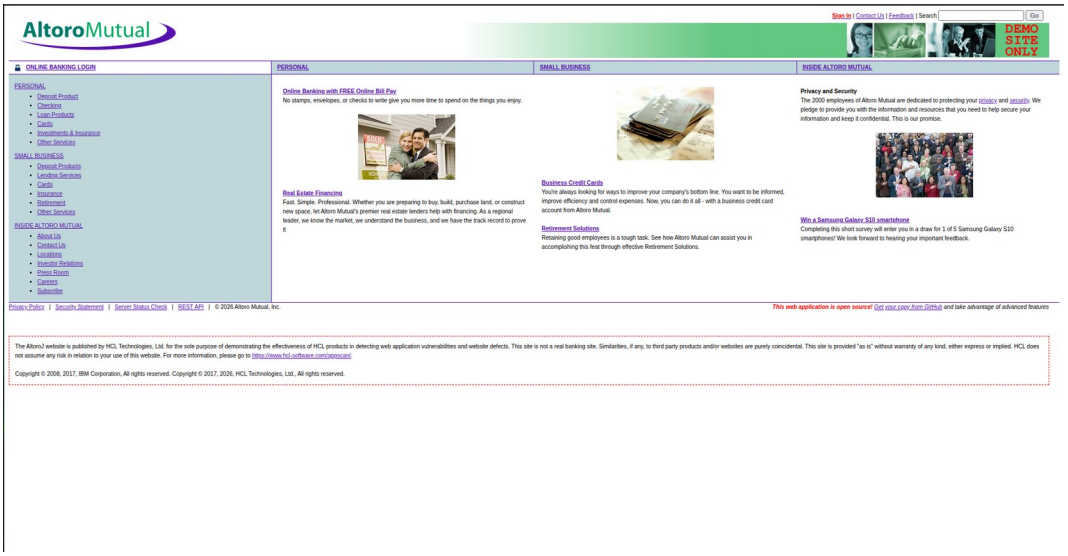
The assessment was conducted within the scope of the authorized demo environment using controlled, non-destructive test inputs. The findings and recommendations are intended to demonstrate common web application security risks and identify appropriate measures for improving application security.

2. Website Overview

Task 1:

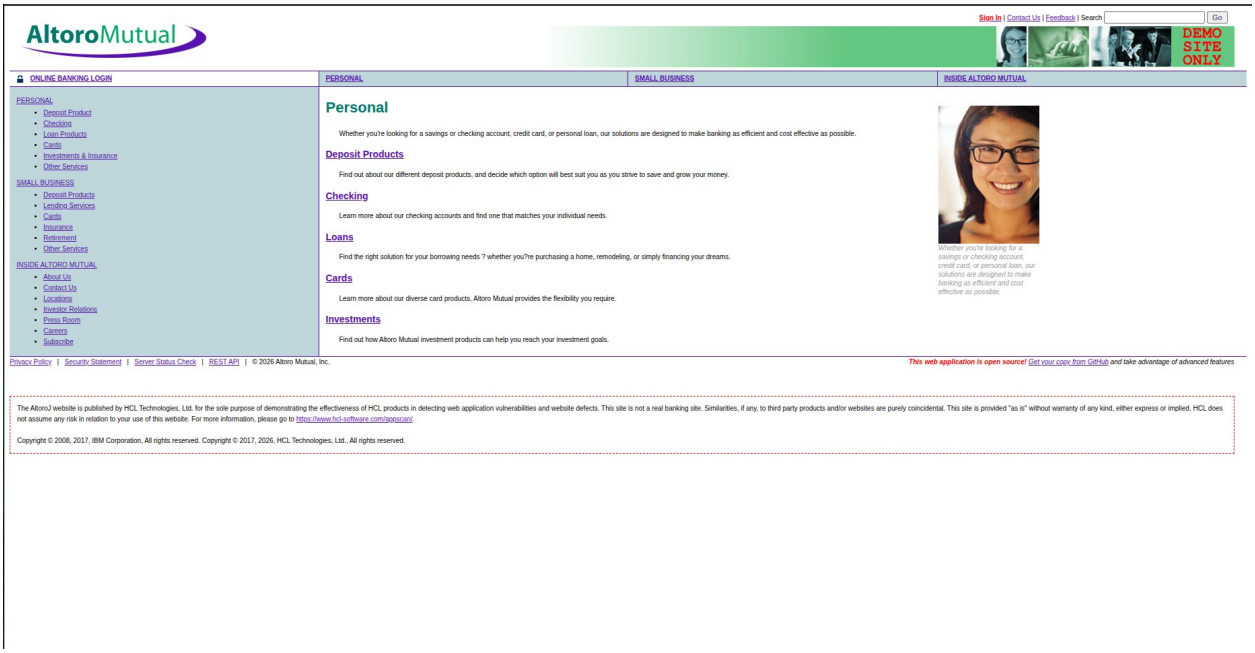
Homepage:

https://demo.testfire.net/



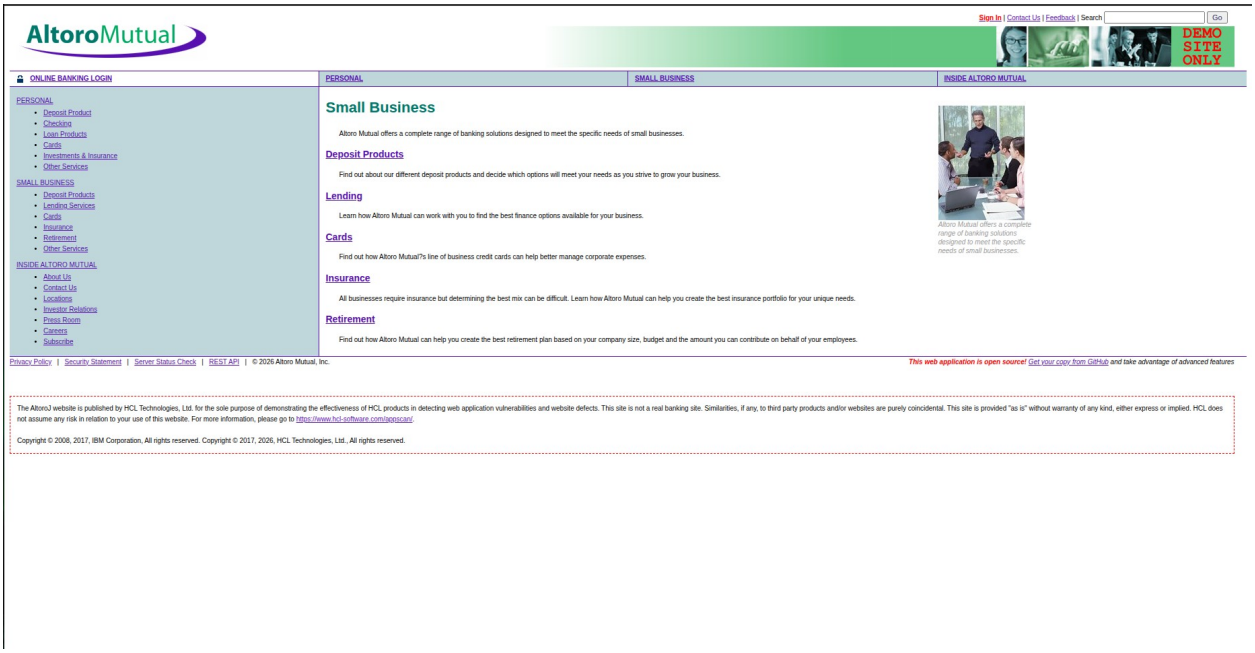
Personal Page:

https://demo.testfire.net/index.jsp?content=personal.htm



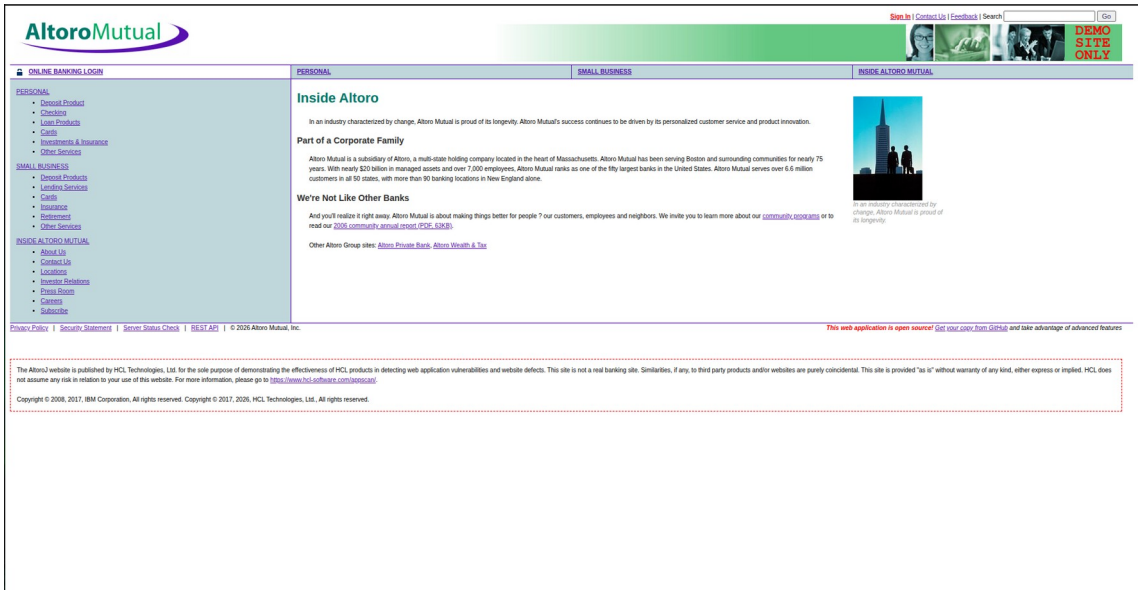
Small Business:

https://demo.testfire.net/index.jsp?content=business.htm



Inside Altro Mutual:

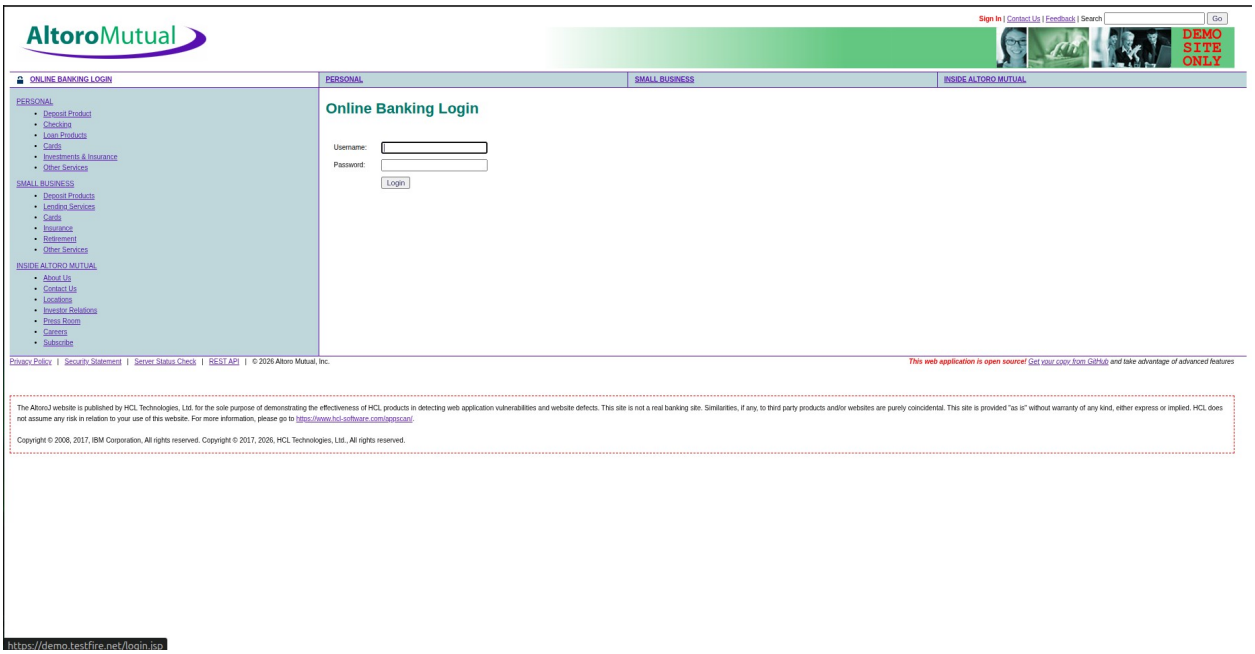
https://demo.testfire.net/index.jsp?content=inside.htm



=> Sign-in, Search and feedback

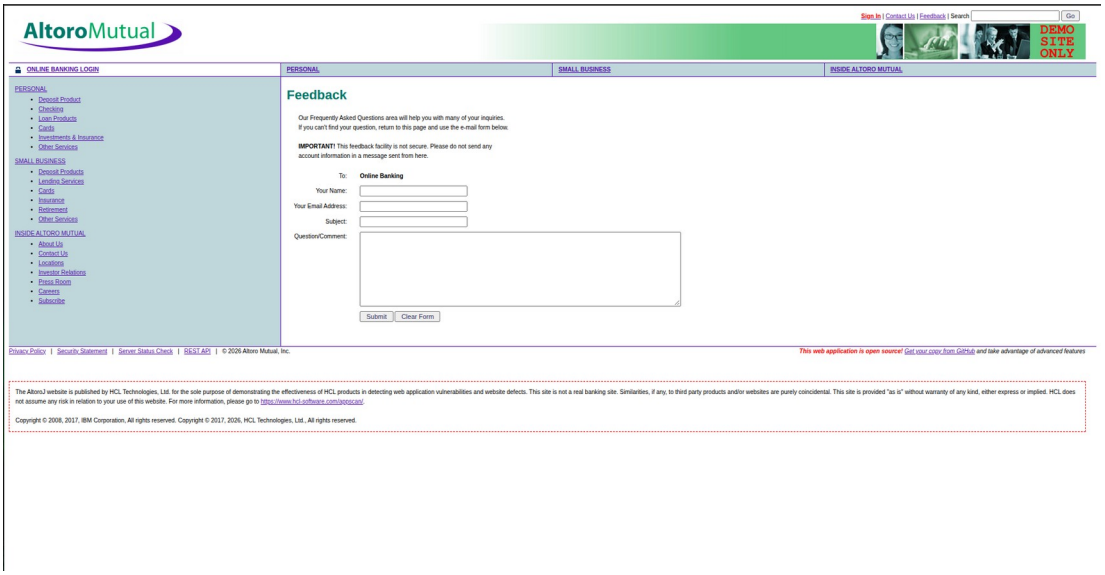
Sign-in Page:

https://demo.testfire.net/login.jsp

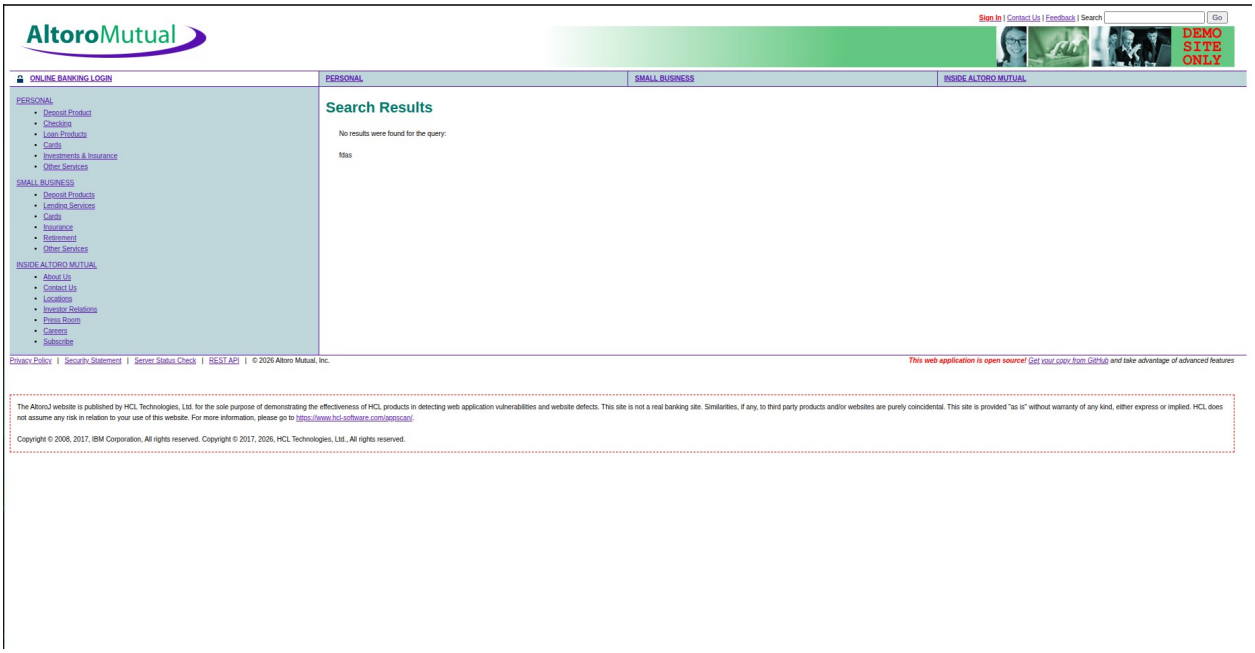


Feedback

https://demo.testfire.net/feedback.jsp



Search:



2.1 Five Important Website Features

1. **Navigation Menu:** Provides links to different sections of the Altoro Mutual website, helping users navigate between pages.
2. **Personal Banking Section:** Presents information about banking products and services intended for individual customers.
3. **Small Business Section:** Provides information about banking services and products for business customers.
4. **Online Banking / Sign In:** Provides a login interface where users can enter their credentials to access online banking.
5. **Inside Altoro Mutual:** Provides company-related information about Altoro Mutual.

2.2 Locations of Sign In, Search, and Feedback

Sign In: The Sign In functionality is located at the top right side. And the url is https://demo.testfire.net/login.jsp

Search: The Search functionality is located at the top right corner where a small input box is present with the label search.

Feedback: The Feedback functionality is located just before the search field. And here's the url: https://demo.testfire.net/feedback.jsp

2.3 Securty Relevance of the Three Functionalities

Sign In: Login forms process usernames and passwords and determine whether a user can access an account. Security testing is important to identify weaknesses in authentication, input handling, and access control that could permit unauthorized access.

Search: Search functionality processes user-supplied input and may display search results or reflect the submitted text on a page. Testing helps identify unsafe database-query construction and improper output encoding, which can lead to SQL injection or cross-site scripting vulnerabilities.

Feedback: Feedback forms accept user-submitted content that may be processed, stored, or displayed by the application. Security testing helps identify unsafe input handling and output rendering that could almmlow injection attacks or expose other users to malicious content.

3. Testing Methodology

Baseline tests were performed using normal inputs. The Sign In feature was tested for SQL injection using controlled inputs, while the Search feature was tested for XSS using an HTML marker and a JavaScript alert payload. The responses were recorded through screenshots in the authorized demo environment.

4. Tools Used

Web Browser: To access and test the application.

Altoro Mutual Demo Website: The authorized testing target.

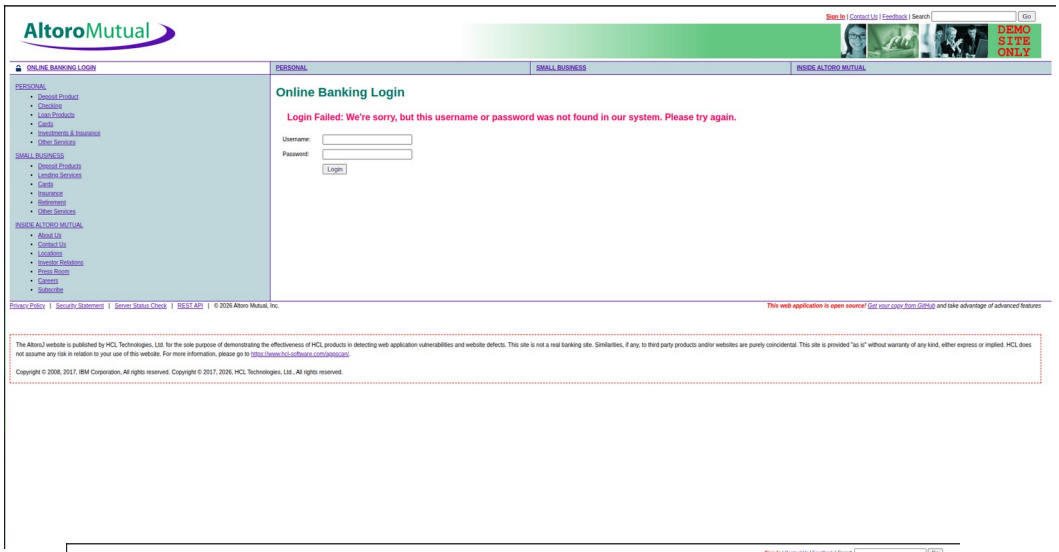
Screenshot Utility: To capture test inputs and results.

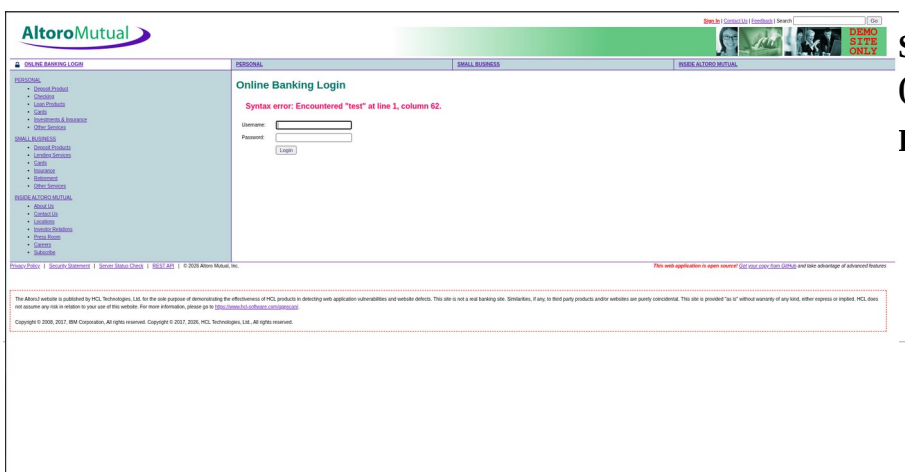
Task 2:

Environment Used: Altoro Mutual

Baseline Test

(Figure 2.1)



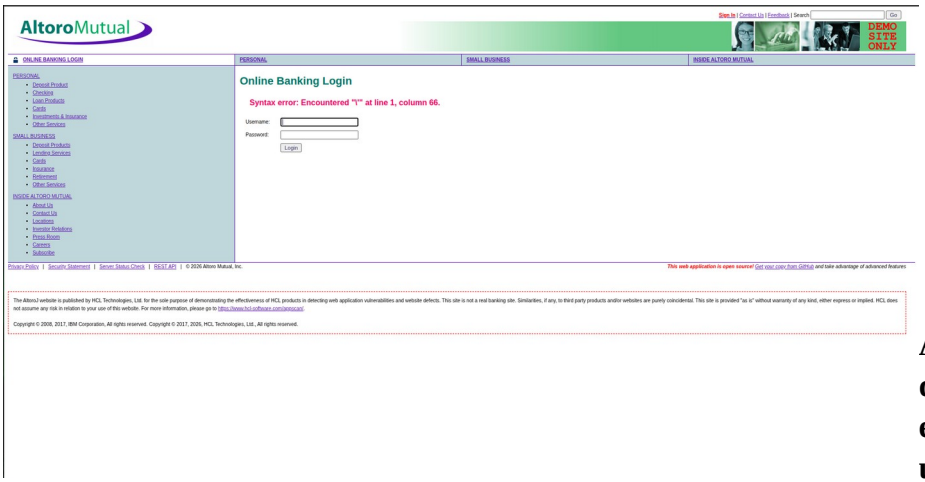


Single-quote probe (username):

Figure (2.2)

Single-quote probe (password):

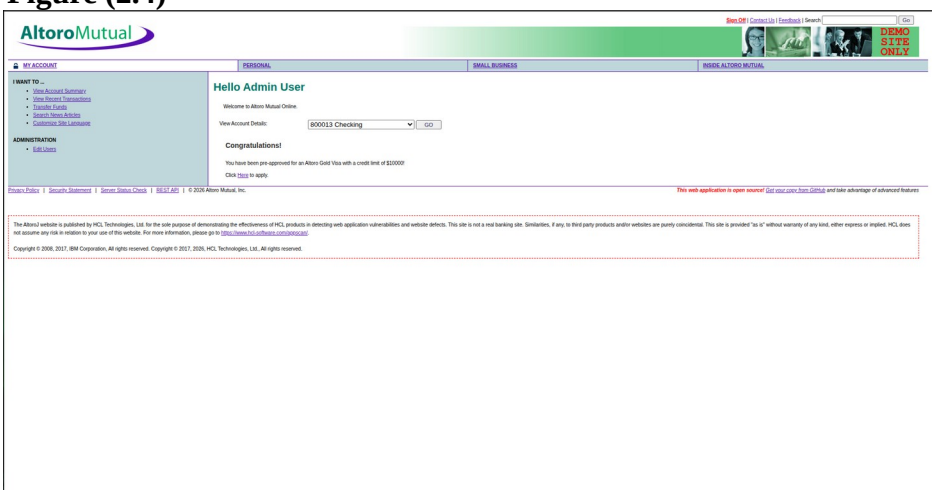
Figure (2.3)



After running these query in username I entered the admin user page,

query used: ' OR '1'='1' --

Figure (2.4)



Analysis SQL Injection Investigation

Q1. Did any test produce a result different from your baseline? Describe the exact difference.

Yes, the results were different from the baseline. When I entered **test** as the username and password, the website showed a normal login failure message. However, when I entered a

single quote (') in the username and password fields separately, the website displayed SQL syntax errors. In the optional test, I entered `' OR '1'='1' --` and the website allowed me to log in and redirected me to the admin user page without valid administrator credentials.

2. Based on what you directly observed, would you classify this functionality as Vulnerable, Not Vulnerable, or Inconclusive? Justify your answer using your own evidence.

I would classify the Sign In functionality as **Vulnerable**. The single quote tests showed SQL syntax errors, and the optional test allowed me to bypass the login. These results suggest that the website is not handling user input safely and that the login process may be vulnerable to SQL injection.

3. What would a real attacker potentially be able to do if this type of weakness existed in a production system?

If the same weakness existed in a real banking website, an attacker might be able to bypass the login and access accounts or features without permission. Depending on the system, they could potentially view sensitive customer information or make unauthorized changes. This could put users' privacy and account security at risk. And also they could be able to steal money.

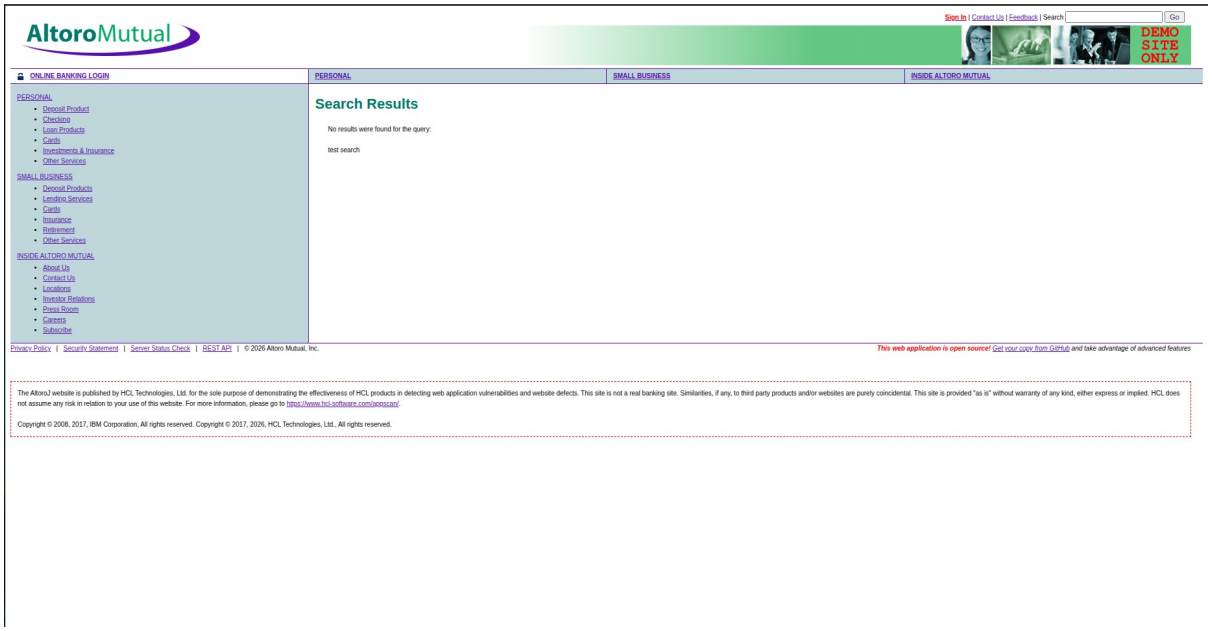
Task 3:

1. Selected Input Point: Search Field

I chose the Search function on the Altoro Mutual website because it accepts user-provided text and processes it to perform a search. I used this input point to investigate whether submitted text is reflected on the results page and whether the application handles HTML input safely.

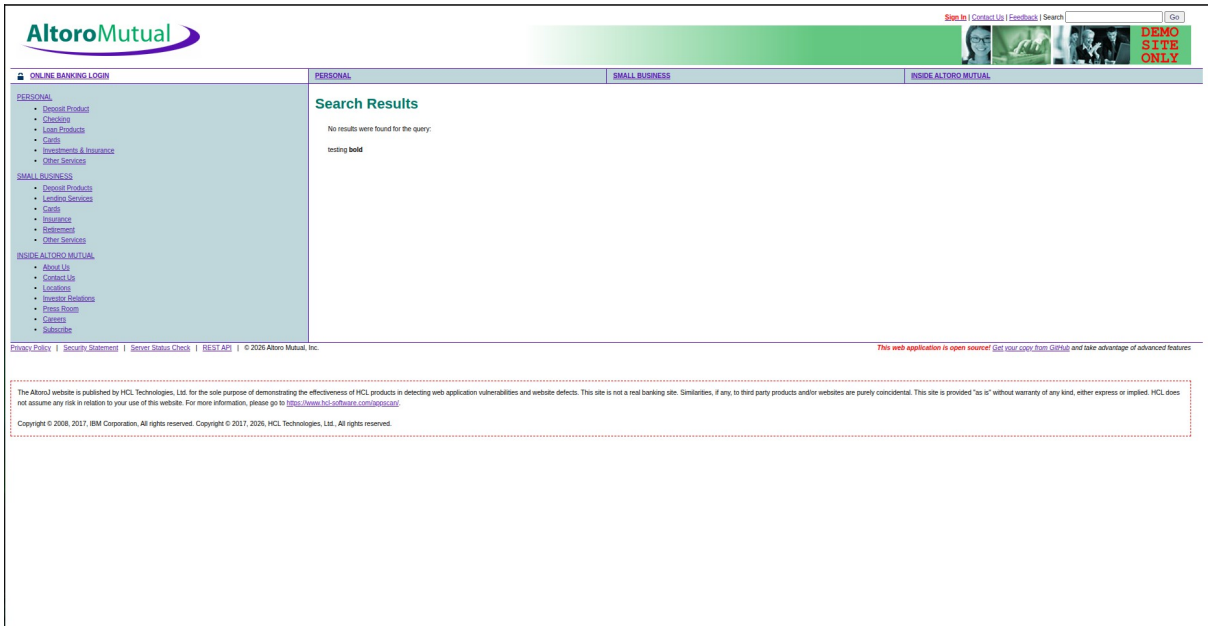
3.1 Baseline Test

Figure (3.1)



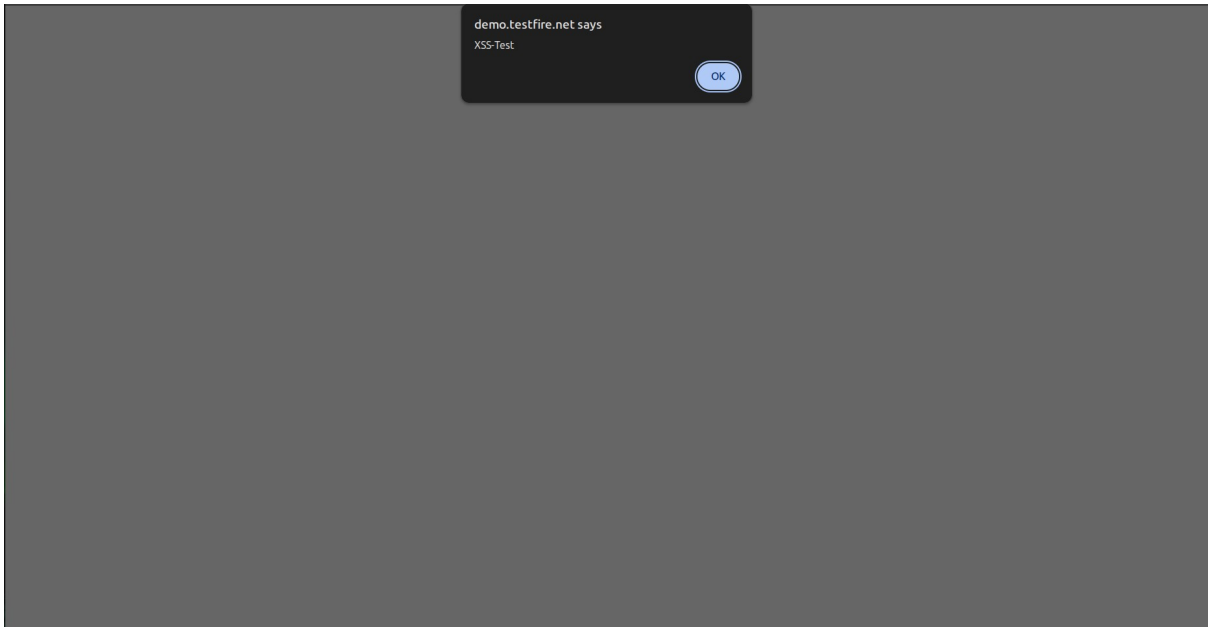
3.2 HTML-marker test:

Figure (3.2)



3.3 Standard alert() test:

Figure (3.3)



Analysis

1. The search input was reflected on the results page, indicating reflected XSS behavior.
2. The HTML marker rendered the word **bold** in bold formatting, and the JavaScript alert test successfully displayed a popup, confirming that the input was actively interpreted.
3. Based on this evidence, the Search field is classified as **Vulnerable**, because the injected JavaScript executed in the browser. Stored XSS was not tested.

Task 4: Security Analysis

4.1 Difference Between SQL Injection and XSS

SQL Injection targets database queries and may allow unauthorized access to data. XSS targets users' browsers by executing injected scripts. In my tests, the login bypass indicated SQL Injection, while the alert popup confirmed XSS.

4.2 Which parts of the CIA triad (Confidentiality, Integrity, Availability) may be affected by SQL injection, and how?

SQL Injection can affect all three parts:

- **Confidentiality:** Exposing private data.
- **Integrity:** Changing or deleting records.
- **Availability:** Disrupting the application or database.

These are possible impacts. I directly observed a login bypass during my test.

4.3 Why is server-side authorization important in banking applications, even when client-side checks already exist?

Client-side checks can be bypassed, so they are not enough. The server must verify user permissions before allowing access to accounts or sensitive banking operations.

4.4 Explain how parameterized queries help prevent SQL injection.

Parameterized queries treat user input as data instead of SQL commands. This prevents specially crafted input from changing the query's logic.

4.5 Explain how context-appropriate output encoding helps prevent XSS.

Output encoding makes user input display as text instead of executable HTML or JavaScript. In my test, the injected HTML and JavaScript were interpreted, so proper output encoding is needed to prevent this.

Analysis Table:

Test ID	Feature Tested	Security Test	Observation	Evidence	Potential Impact	Recommended Fix	Result
SQL-01	Sign In	Baseline: `test` / `test`	**Login Failed: We're sorry, but this username or password was not found in our system. Please try again.**	**(Figure 2.1)**	No vulnerability demonstrated by this test; establishes baseline behavior.	Use generic authentication error messages.	Pass
SQL-02	Sign In	Username: `'`	**Syntax error: Encountered "d" at line 1, column 62.**	**(Figure 2.2)**	Error disclosure may reveal implementation details.	Parameterized queries and generic error handling.	Fail
SQL-03	Sign In	Password: `'`	**Syntax error: Encountered "\'" at line 1, column 66.**	**(Figure 2.3)**	Error disclosure may help attackers investigate the application.	Parameterized queries and generic error handling.	Fail
SQL-04	Sign In	Conceptual SQL injection test	The application accepted the login attempt and redirected me to the admin user page without valid administrator credentials.	**(Figure 2.4)**	Unauthorized access to protected functionality and potentially sensitive data.	Parameterized queries, secure authentication, least privilege.	Fail
XSS-01	Search	Baseline: test search	Search input was reflected	**(Figure 3.1)**	Depends on whether input is	Context-appropriate output encoding.	Inconclusive

					reflected unsafely.		
XSS-02	Search	HTML marker: `testing<b>bold</b>`	Render acctual html tag, it actually made bold as **bold**	**(Figure 3.2)**	Unsafe HTML interpretation may allow page manipulation; this alone does not prove XSS.	Context-appropriate output encoding and safe templating.	fail
XSS-03	Search	Script test: `<script>alert('XSS-Test')</script>`	It displayed the alert pop up which means it executed the javascript	**(Figure 3.3)**	Executable injected script may affect visitors viewing the vulnerable page.	Output encoding, safe DOM handling.	fail

Conclusion

The assessment identified SQL injection in the Sign In feature and reflected XSS in the Search feature. These findings demonstrate the risks of improper input handling. Parameterized queries and appropriate output encoding can help prevent these vulnerabilities and improve web application security.